

#중급반 5주차

너비 우선 탐색 (BFS)

<Breadth First Search>

2p /<복습 - 그래프>

6p /<복습 - DFS>

16p /<BFS>

30p /<연습 문제>

#BFS

<복습 - 그래프>

그래프를 표현하는 방식으로는 대표적으로 2가지가 있습니다.

인접 리스트 (Adjacency List) 와 인접 행렬 (Adjacency Matrix)

1. 인접 리스트는 각각 정점들에 대해 간선으로 연결되어 있는 정점들을 각각의 리스트로 표현하는 방식입니다.
2. 인접 행렬은 i 번 정점에서 j 번 간선으로 가는 정점이 존재할 경우 행렬의 i 번째 행 j 번째 열의 값을 1, 존재하지 않을 경우 0으로 설정해 표현하는 방식입니다.

#BFS

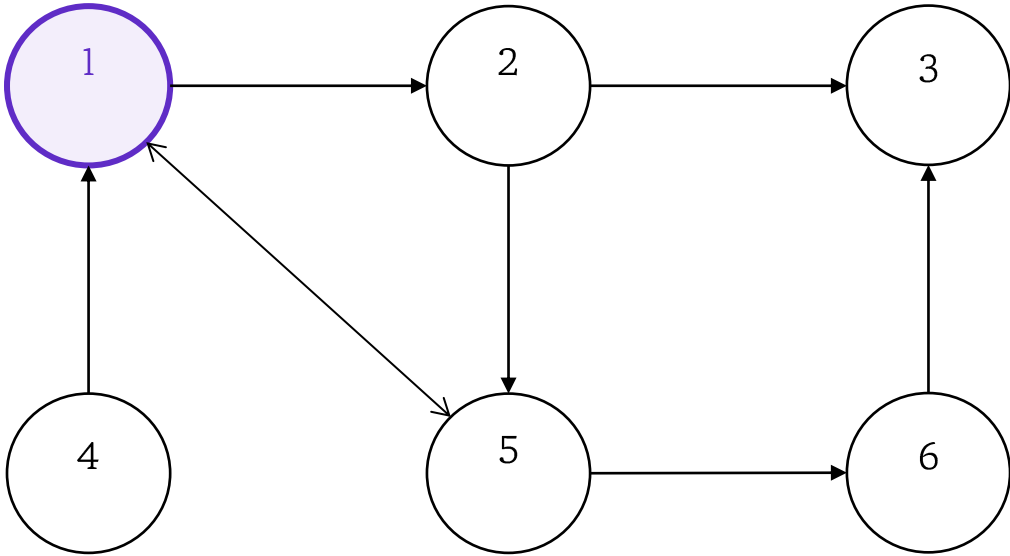
<복습 - 그래프>

<인접 리스트>

1	2, 5
2	3, 5
3	
4	1
5	1, 6
6	3

<인접 행렬>

0	1	0	0	1	0
0	0	1	0	1	0
0	0	0	0	0	0
1	0	0	0	0	0
1	0	0	0	0	1
0	0	1	0	0	0



#BFS

<복습 - 그래프>

C++로 인접 리스트를 어떻게 구현할 수 있는지
알아봅시다.

인접 리스트의 구현 자체는 한 줄로 충분합니다.
최대 정점 개수 만큼의 정수형 vector 배열을
만들어 주면 됩니다.

정점 u에서 정점 v로 가는 간선을 추가하기
위해서는 u번째 vector에 v를 추가해주면 됩니다.

```
vector<int> edges[200005];  
  
int u, v;  
cin >> u >> v;  
edges[u].push_back(v);
```

#BFS

<복습 - 그래프>

다음은 인접 행렬의 구현을 알아보시다.

인접 행렬의 구현도 간단히 2차원 배열 하나로 끝납니다.

정점 u 에서 정점 v 로 가는 간선을 추가하기 위해서는 u 번째 행 v 번째 열을 1로 바꾸면 됩니다.

```
int graph[305][305];
```

```
int u, v;  
cin >> u >> v;  
graph[u][v] = 1;
```

#BFS

<복습 - DFS>

DFS/Depth-First-Search - 깊이 우선 탐색

1. 깊이 우선 탐색은 그래프에서 **한 가지로 최대한 깊이 들어가서 탐색**한후 더 이상 탐색할 수 없을 경우 돌아와서 다음 가지를 탐색하는 방식입니다.

#BFS

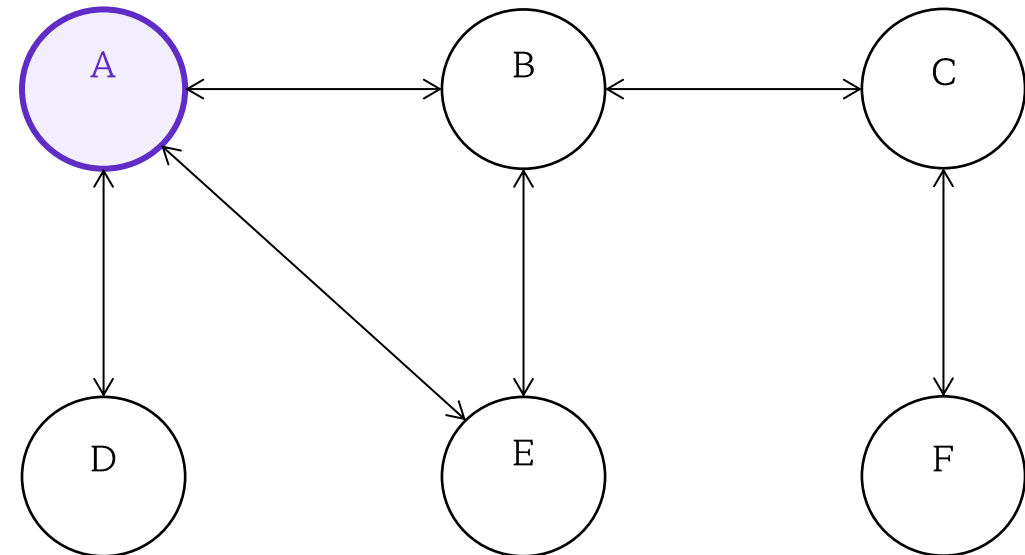
<복습 - DFS>

오른쪽 무방향 그래프를 가지고 한번 이해해보도록 합시다.

엄밀성을 위해,

- 시작 정점은 A입니다.
 - 탐색할 수 있는 정점이 여러 개일 경우 사전 순으로 가장 빠른 정점을 선택합니다.
 - 한번 방문한 정점은 다시 방문하지 않습니다.
- 라고 합시다.

이 경우 방문 순서는 A B C F E D 가 됩니다.



#BFS

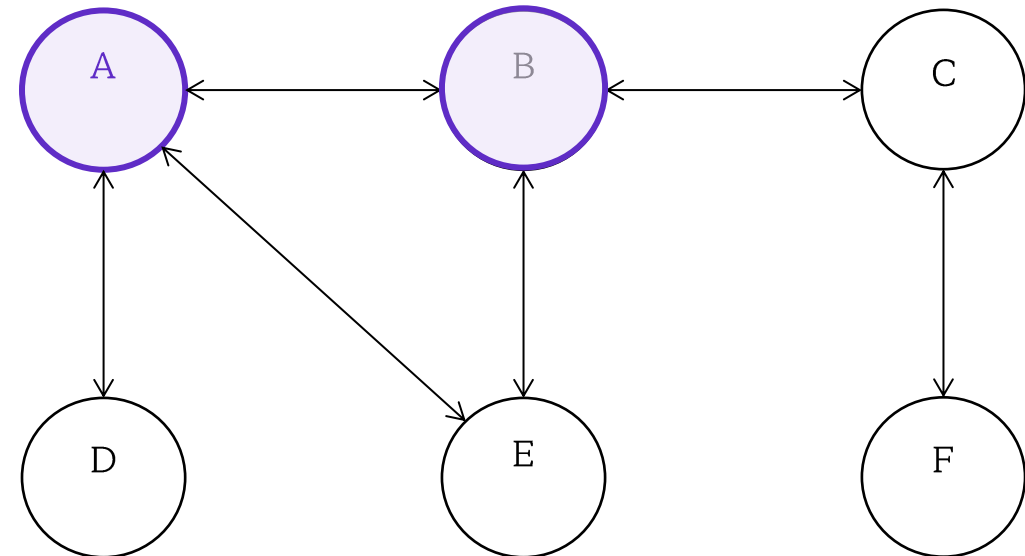
<복습 - DFS>

오른쪽 무방향 그래프를 가지고 한번 이해해보도록 합시다.

엄밀성을 위해,

- 시작 정점은 A입니다.
 - 탐색할 수 있는 정점이 여러 개일 경우 사전 순으로 가장 빠른 정점을 선택합니다.
 - 한번 방문한 정점은 다시 방문하지 않습니다.
- 라고 합시다.

이 경우 방문 순서는 A B C F E D 가 됩니다.



#BFS

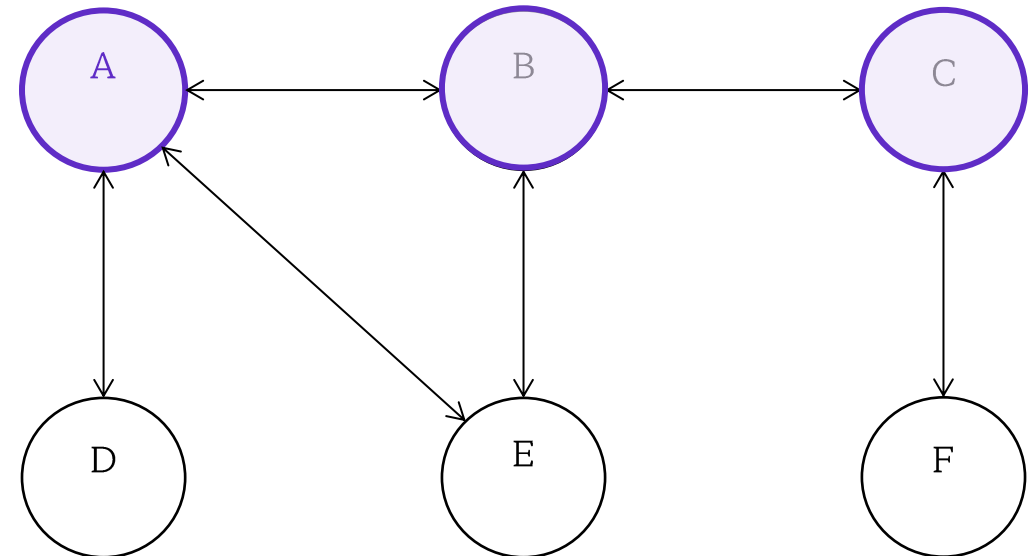
<복습 - DFS>

오른쪽 무방향 그래프를 가지고 한번 이해해보도록 합시다.

엄밀성을 위해,

- 시작 정점은 A입니다.
 - 탐색할 수 있는 정점이 여러 개일 경우 사전 순으로 가장 빠른 정점을 선택합니다.
 - 한번 방문한 정점은 다시 방문하지 않습니다.
- 라고 합시다.

이 경우 방문 순서는 A B C F E D 가 됩니다.



#BFS

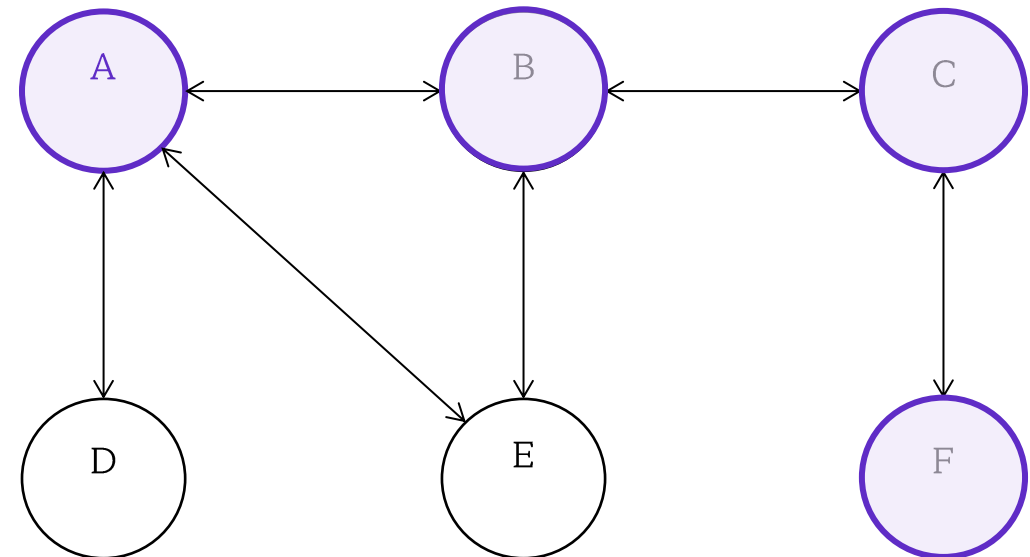
<복습 - DFS>

오른쪽 무방향 그래프를 가지고 한번 이해해보도록 합시다.

엄밀성을 위해,

- 시작 정점은 A입니다.
 - 탐색할 수 있는 정점이 여러 개일 경우 사전 순으로 가장 빠른 정점을 선택합니다.
 - 한번 방문한 정점은 다시 방문하지 않습니다.
- 라고 합시다.

이 경우 방문 순서는 A B C F E D 가 됩니다.



#BFS

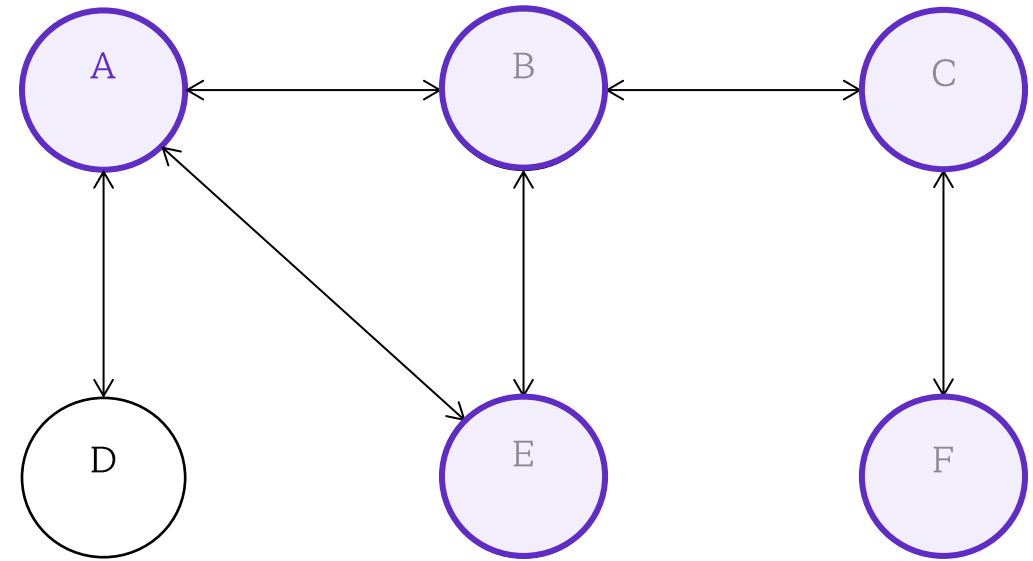
<복습 - DFS>

오른쪽 무방향 그래프를 가지고 한번 이해해보도록 합시다.

엄밀성을 위해,

- 시작 정점은 A입니다.
 - 탐색할 수 있는 정점이 여러 개일 경우 사전 순으로 가장 빠른 정점을 선택합니다.
 - 한번 방문한 정점은 다시 방문하지 않습니다.
- 라고 합시다.

이 경우 방문 순서는 A B C F E D 가 됩니다.



#BFS

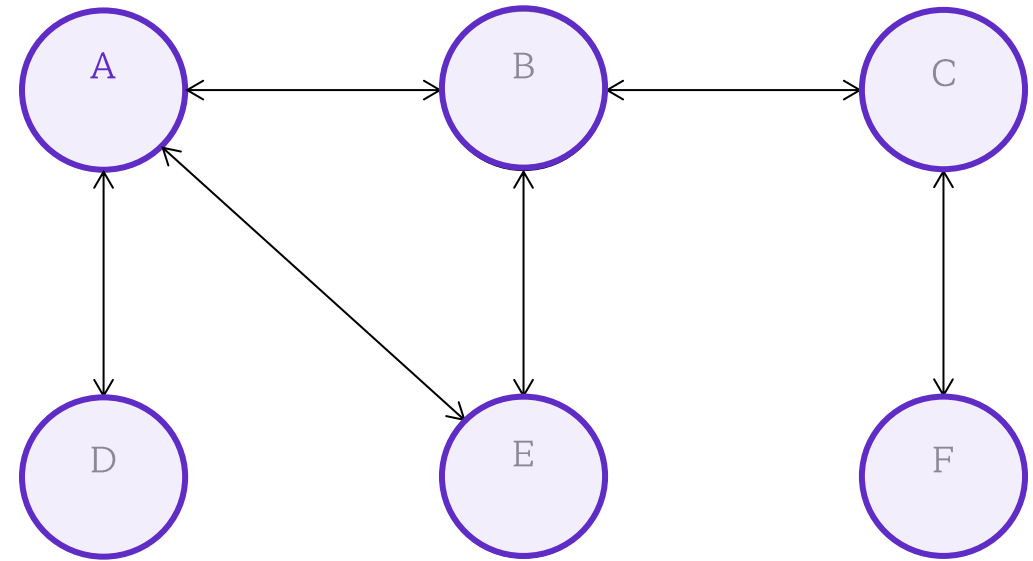
<복습 - DFS>

오른쪽 무방향 그래프를 가지고 한번 이해해보도록 합시다.

엄밀성을 위해,

- 시작 정점은 A입니다.
 - 탐색할 수 있는 정점이 여러 개일 경우 사전 순으로 가장 빠른 정점을 선택합니다.
 - 한번 방문한 정점은 다시 방문하지 않습니다.
- 라고 합시다.

이 경우 방문 순서는 A B C F E D 가 됩니다.



#BFS

<복습 - DFS>

이제 실제로 DFS를 구현해볼 차례입니다.

DFS는 재귀함수를 이용해 재귀적으로 구현합니다.

```
bool vis[1005];  
vector<int> edges[1005];  
void dfs(int cur)  
{  
    vis[cur] = 1;  
    cout << cur << ' ' ;  
    for (int &nxt : edges[cur])  
    {  
        if (vis[nxt]) continue;  
        dfs(nxt);  
    }  
}
```

#BFS

<복습 - DFS>

오른쪽은 간단한 재귀 DFS 구현입니다.
DFS로 방문한 순서대로 정점 번호를 출력합니다.

방문했던 정점은 다시 방문하지 않기 위해,
`vis`라는 이름의 bool 배열을 만들어줍니다.
`vis[i]`의 값이 true면 i번째 정점을 이미 방문했다는
뜻입니다.

이제 천천히 dfs 함수를 살펴보도록 합시다.
이 함수의 인자는 현재 정점 번호(`cur`)입니다.
우선 현재 방문한 정점을 방문했다고 기록해줍니다.
그 후 그 정점 번호를 출력합니다.

```
bool vis[1005];  
  
vector<int> edges[1005];  
  
void dfs(int cur)  
{  
    vis[cur] = 1;  
    cout << cur << ' ';  
    for (int &nxt : edges[cur])  
    {  
        if (vis[nxt]) continue;  
        dfs(nxt);  
    }  
}
```

#BFS

<복습 - DFS>

그 후, 그 정점과 연결된 정점들을 확인합니다.

만약 이미 방문한 정점이라면,
더 이상 탐색할 수 없으므로 continue합니다.

방문하지 않은 정점이라면,
그 가지로 더 깊이 가기 위해 재귀 호출 인자로
그 정점의 번호를 넘겨줍니다.

```
bool vis[1005];  
vector<int> edges[1005];  
  
void dfs(int cur)  
{  
    vis[cur] = 1;  
    cout << cur << ' ' << '\n';  
    for (int &nxt : edges[cur])  
    {  
        if (vis[nxt]) continue;  
        dfs(nxt);  
    }  
}
```

#BFS

<BFS>

BFS/Breadth-First-Search – 너비 우선 탐색

1. 너비 우선 탐색은 시작 정점으로부터 가장 인접한 정점을 먼저 탐색하는 방식입니다.
2. 거리에 따라 단계별로 그 단계에 속하는 정점들을 방문한다고 생각하면 이해하기 편할 수 있습니다.

#BFS

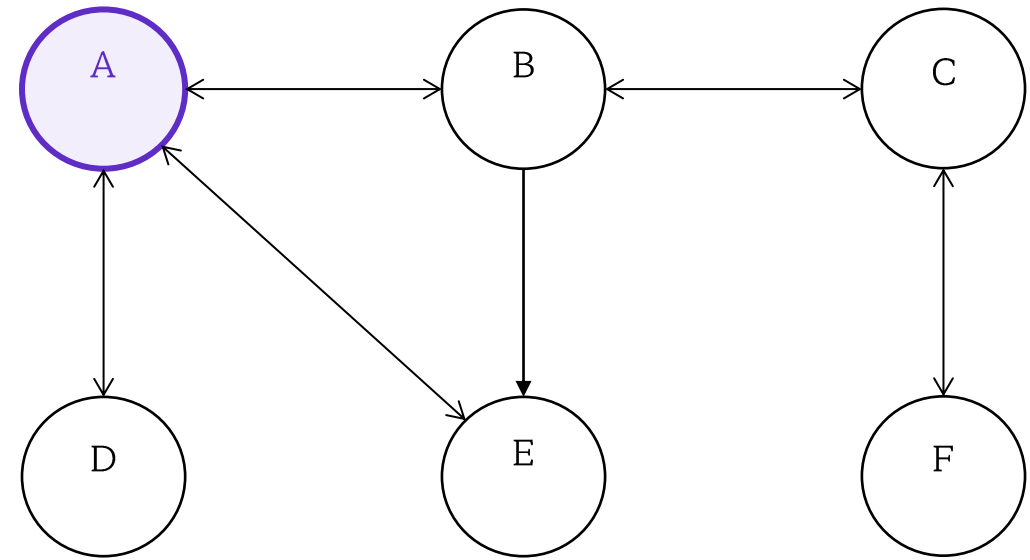
<BFS>

이번에도 오른쪽 무방향 그래프를 가지고 한번 이해해보도록 합시다.

엄밀성을 위해,

- 시작 정점은 A입니다.
 - 탐색할 수 있는 정점이 여러 개일 경우 사전 순으로 가장 빠른 정점을 선택합니다.
 - 한번 방문한 정점은 다시 방문하지 않습니다.
- 라고 합시다.

이 경우 방문 순서는 A B D E C F가 됩니다.



#BFS

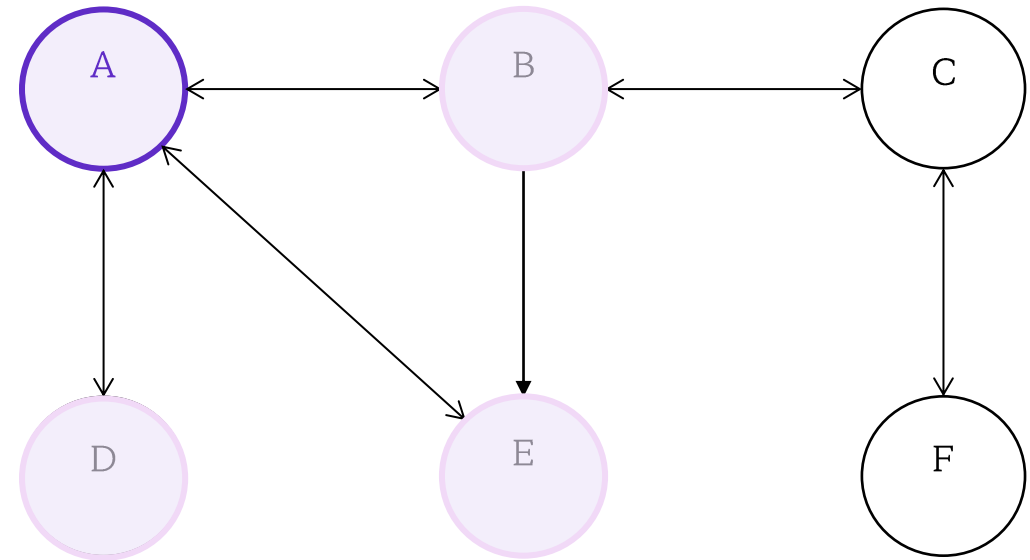
<BFS>

이번에도 오른쪽 무방향 그래프를 가지고 한번 이해해보도록 합시다.

엄밀성을 위해,

- 시작 정점은 A입니다.
 - 탐색할 수 있는 정점이 여러 개일 경우 사전 순으로 가장 빠른 정점을 선택합니다.
 - 한번 방문한 정점은 다시 방문하지 않습니다.
- 라고 합시다.

이 경우 방문 순서는 A B D E C F가 됩니다.



#BFS

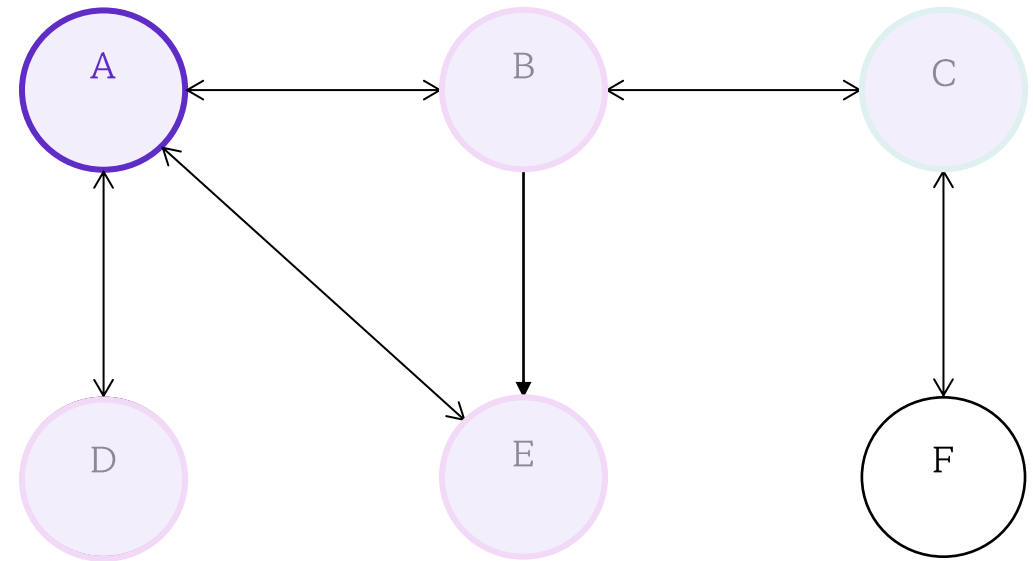
<BFS>

이번에도 오른쪽 무방향 그래프를 가지고 한번 이해해보도록 합시다.

엄밀성을 위해,

- 시작 정점은 A입니다.
 - 탐색할 수 있는 정점이 여러 개일 경우 사전 순으로 가장 빠른 정점을 선택합니다.
 - 한번 방문한 정점은 다시 방문하지 않습니다.
- 라고 합시다.

이 경우 방문 순서는 A B D E C F가 됩니다.



#BFS

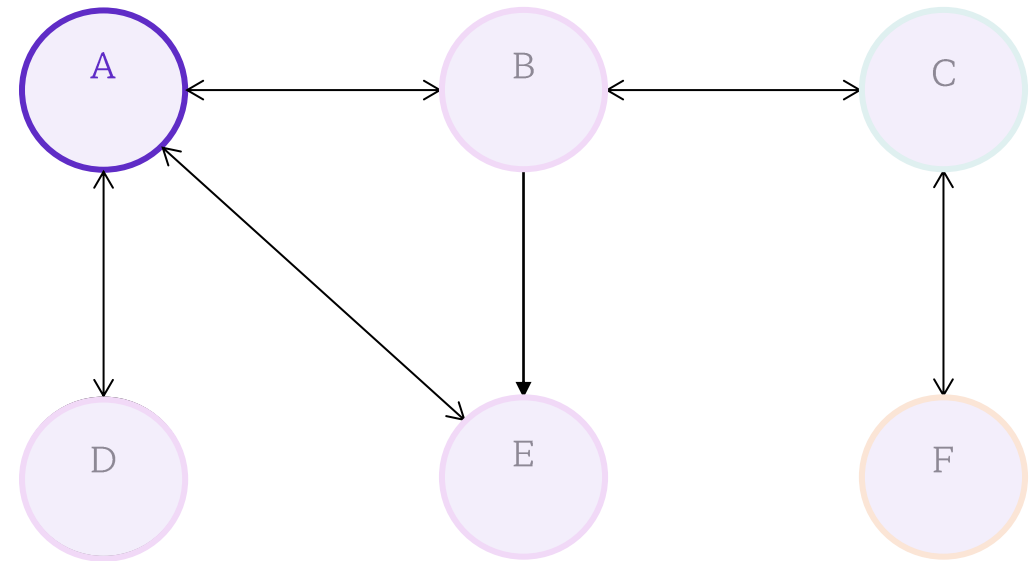
<BFS>

이번에도 오른쪽 무방향 그래프를 가지고 한번 이해해보도록 합시다.

엄밀성을 위해,

- 시작 정점은 A입니다.
 - 탐색할 수 있는 정점이 여러 개일 경우 사전 순으로 가장 빠른 정점을 선택합니다.
 - 한번 방문한 정점은 다시 방문하지 않습니다.
- 라고 합시다.

이 경우 방문 순서는 A B D E C F가 됩니다.



#BFS

<BFS>

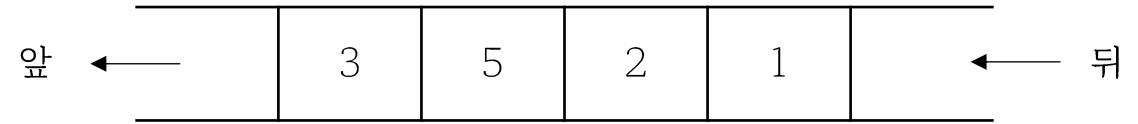
BFS를 구현할 때에는 Queue라는 자료 구조를 사용합니다.

Queue는 FIFO (First-In-First-Out) 형태의 자료 구조입니다. 즉, 먼저 삽입된 원소는 항상 먼저 삭제됩니다.

Queue에는 앞과 뒤가 있습니다.

Queue의 앞에서는 조회 및 삭제가 가능합니다.

Queue의 뒤에서는 삽입이 가능합니다.



#BFS

<BFS>

BFS를 구현해봅시다.

DFS와 다르게, BFS는 재귀적으로 구현하지 않고, queue를 사용해 비재귀적으로 구현합니다.

queue가 FIFO (First-In-First-Out) 형태의 자료구조임을 생각해보면, 작동 방식을 이해하기 어렵지 않을 것입니다.

```
bool vis[1005];
vector<int> edges[1005];
void bfs(int start)
{
    queue<int> q;

    vis[start] = 1;
    q.push(start);

    while (!q.empty())
    {
        int cur = q.front();
        cout << cur << ' ';
        q.pop();
        for (int &nxt : edges[cur])
        {
            if (vis[nxt]) continue;
            vis[nxt] = true;
            q.push(nxt);
        }
    }
}
```

#BFS

<BFS>

queue에 시작 정점을 넣어줍니다.

그 후 queue가 빌 때 까지 다음 슬라이드에서
설명할 과정을 반복합니다.

```
bool vis[1005];
vector<int> edges[1005];
void bfs(int start)
{
    queue<int> q;

    vis[start] = 1;
    q.push(start);

    while (!q.empty())
    {
        int cur = q.front();
        cout << cur << ' ';
        q.pop();
        for (int &nxt : edges[cur])
        {
            if (vis[nxt]) continue;
            vis[nxt] = true;
            q.push(nxt);
        }
    }
}
```

#BFS

<BFS>

Queue가 빌 때까지(모든 정점을 방문했을 때까지)
다음을 반복합니다.

1. queue의 맨 앞에 있는 정점을 현재 방문한 정점 **cur**에 저장하고 번호를 출력합니다.
2. queue의 맨 앞에 있는 정점을 queue에서 **제거**합니다.
3. cur과 연결된 정점들 중 아직 **방문하지 않은** 정점들을 **queue에 넣어줍니다**.
4. 1~3 과정을 queue가 빌 때 까지 반복합니다.
queue가 비었다는 것은 더 이상 방문할 수 있는 정점이 없다는 뜻이기 때문입니다.

```

bool vis[1005];
vector<int> edges[1005];
void bfs(int start)
{
    queue<int> q;

    vis[start] = 1;
    q.push(start);

    while (!q.empty())
    {
        int cur = q.front();
        cout << cur << ' ';
        q.pop();
        for (int &nxt : edges[cur])
        {
            if (vis[nxt]) continue;
            vis[nxt] = true;
            q.push(nxt);
        }
    }
}

```


#BFS

<BFS>

DFS와 BFS의 시간 복잡도 - $O(V + E)$ $(V = \text{정점의 개수}, E = \text{간선의 개수})$

1. 모든 정점을 한번씩 방문하기 때문에 V
2. 모든 정점에서 연결된 간선을 한번씩 체크하기 때문에
(방문 여부 체크 과정에서) E

#BFS

<BFS>

1 잡기놀이 (JUNGOL #3640)

<문제 설명>

- X에서 X-1, X+1, 2X로 이동할 수 있을 때, N에서 K로 이동하는 최단 거리를 구하여라.

<제약 조건>

- $1 \leq N \leq 100,000$
- $1 \leq K \leq 100,000$

#BFS

<BFS>

1 잡기놀이 (JUNGOL #3640)

<문제 해설>

- X에서 X-1, X+1, 2X로 갈 수 있다는 정보를 간선으로 생각합시다.
- 이러한 그래프에서 N에서 K로의 최단 거리는 어떻게 구할 수 있을까요?
- **가중치가 없는 그래프**에서는, **BFS**를 통해 최단 거리를 구할 수 있습니다.

#BFS

<BFS>

dist 배열이 -1이라면, 아직 방문하지 않음을 나타내고
-1이 아니라면, 시작점과의 거리를 나타냅니다.

시작점인 N을 큐에 넣고 BFS를 시작합니다.

현재 정점이 now고, 연결된 정점 x를 아직 방문하지
않았다면, x의 거리는 now의 거리보다 1 큼니다.

$\text{dist}[x] = \text{dist}[\text{now}] + 1$ 처럼 dist 배열을 업데이트하여,
시작점으로부터의 거리를 저장합니다.

```
int N, K;
cin >> N >> K;
for (int i = 0; i <= max(N, K) + 1; i++) dist[i] = -1;
dist[N] = 0;
queue<int> q;
q.push(N);
while (!q.empty())
{
    int now = q.front();
    q.pop();
    if (now * 2 <= K + 1 and dist[now * 2] == -1)
    {
        dist[now * 2] = dist[now] + 1;
        q.push(now * 2);
    }
    if (now - 1 >= 0 and dist[now - 1] == -1)
    {
        dist[now - 1] = dist[now] + 1;
        q.push(now - 1);
    }
    if (now + 1 <= K + 1 and dist[now + 1] == -1)
    {
        dist[now + 1] = dist[now] + 1;
        q.push(now + 1);
    }
}
cout << dist[K];
```

#BFS

<BFS>

도착 지점이 K이기 때문에,
K+1보다 큰 위치로 갈 필요가 없습니다.

증명)

K보다 큰 위치로 간다면, K에 도달하기 위해서 언젠가는 왼쪽으로 이동해야 합니다.

*2를 사용해 K+2 이상으로 간 뒤 -1을 2번 이상 사용해 K를 가는 것 보다, -1을 먼저 사용한 후 K로 가는 것이 더 빠르기 때문입니다.

비슷하게, 0보다 작은 위치로 갈 필요도 없습니다.

따라서 now의 범위를 확인하여 너무 많이 방문하지 않도록 효율성을 높여줄 수 있습니다.

```
int N, K;
cin >> N >> K;
for (int i = 0; i <= max(N, K) + 1; i++) dist[i] = -1;
dist[N] = 0;
queue<int> q;
q.push(N);
while (!q.empty())
{
    int now = q.front();
    q.pop();
    if (now * 2 <= K + 1 and dist[now * 2] == -1)
    {
        dist[now * 2] = dist[now] + 1;
        q.push(now * 2);
    }
    if (now - 1 >= 0 and dist[now - 1] == -1)
    {
        dist[now - 1] = dist[now] + 1;
        q.push(now - 1);
    }
    if (now + 1 <= K + 1 and dist[now + 1] == -1)
    {
        dist[now + 1] = dist[now] + 1;
        q.push(now + 1);
    }
}
cout << dist[K];
```



#필수문제

1 장기 2 (JUNGOL #4189)

BFS는 최단 거리를 구할 때 쓸 수 있습니다.

1 안전 영역 (JUNGOL #2298)

연결 요소의 개수 구하기

1 잡기놀이 (JUNGOL #3640)

강의에서 설명한 문제입니다.

#연습 문제 도전

1 영역 구하기 (JUNGOL #1457)

최단 거리를 구할 수 있으니 이 문제도 해결할 수 있습니다.

5 토마토(고) (JUNGOL #2613)

2차원 격자 그래프 위에서의 BFS 탐색

4 토마토(초) (JUNGOL #2606)

3차원 격자 그래프 위에서의 BFS 탐색

1 저글링 방사능 오염 (JUNGOL #1078)

BFS는 최단 거리를 구할 때 쓸 수 있습니다.

5 보물섬 (JUNGOL #1462)

문제 조건에 주목합시다

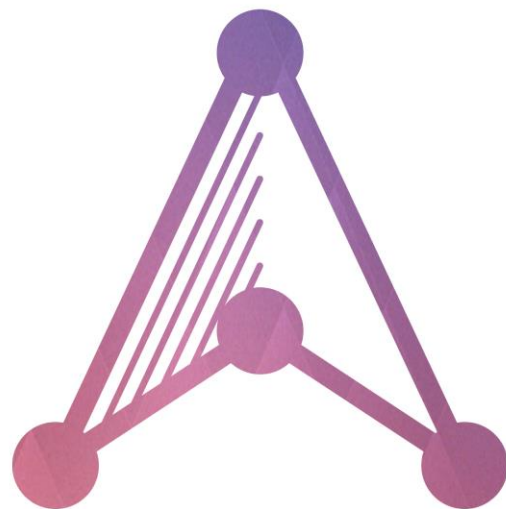
#연습 문제 도전

4 화염에서탈출 (JUNGOL #1082)

Multi-Source BFS

3 로봇 (JUNGOL #1006)

‘상태’란 무엇일까요?



A L O H A
The algorithm club.